

Lab 1 - Robot Operating System: Part A

1.1 Workspace setup for Gazebo simulator

Both Ubuntu 22.04 and ROS humble (ROS2) have already been installed in the Lab PCs. You should open a terminal using the keyboard input (Ctrl + Alt + T), in the home directory.

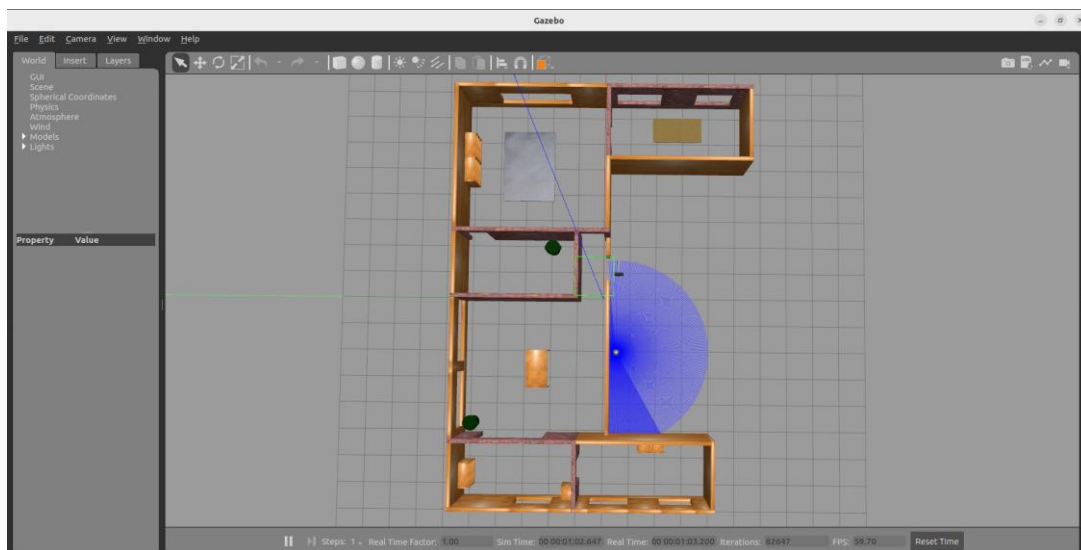
1. Typing the following commands one by one to prepare the repository:

```
source /opt/ros/humble/setup.bash
cd ~/M-Drive/
mkdir -p ./ros2_ws/src
cd ros2_ws
colcon build
```

2. You can use the following commands to run Gazebo simulator:

```
source ~/M-Drive/ros2_ws/install/setup.sh
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

In few minutes, you will see the following picture in the simulation world. You may need to restart Gazebo for the robot to spawn:



3. You can use `ctl^C` to stop the display of the simulation world above.

1.2 Creating new package.

In ROS2, if you want to install your code, you will organise it in a package. A node is an executable (C++ program, Python program, etc.) in packages. ROS2 uses *ament* as its build system and *colcon* as its build tool.

1. Create packages in *src* folder in your workspace.

```
cd ~/M-Drive/ros2_ws/src
```

2. Using the command below to create a package *tutorial_pkg* with a node *tutorial_pkg_node*.

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 --node-name tutorial_pkg_node tutorial_pkg
```

Now you will have *tutorial_pkg* package in your *src* folder.

Inside *tutorial_pkg* directory, *CMakeLists.txt* describes how to build the code within the package and *package.xml* contains the meta information about the package.

1.3 Write your code.

1. Now you can write the code to the file *tutorial_pkg_node.cpp*.

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

2. Delete the existing code in file and copy the below code to the file.

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
#include "geometry_msgs/msg/twist.hpp"

using namespace std::chrono_literals;

class Stopper : public rclcpp::Node{
public:
    /* velocity control variables*/
    constexpr const static double FORWARD_SPEED_LOW = 0.1;
    constexpr const static double FORWARD_SPEED_MIDDLE = 0.3;
    constexpr const static double FORWARD_SPEED_HIGH = 0.5;
    constexpr const static double FORWARD_SPEED_STOP = 0;
    constexpr const static double TURN_LEFT_SPEED_LOW = 0.3;
    constexpr const static double TURN_LEFT_SPEED_MIDDLE = 0.6;
    constexpr const static double TURN_LEFT_SPEED_HIGH = 1.0;
    constexpr const static double TURN_RIGHT_SPEED_LOW = -0.3;
    constexpr const static double TURN_RIGHT_SPEED_MIDDLE = -0.6;
    constexpr const static double TURN_RIGHT_SPEED_HIGH = -1.0;

    /* class constructor */
    Stopper():Node("Stopper"), count_(0){
        publisher_ = this->create_publisher<geometry_msgs::msg::Twist>("cmd_vel", 10);};

    /* moving function */
    void startMoving();
    void moveStop();
    void moveForward(double forwardSpeed);
    void moveRight(double turn_right_speed);

private:
    // Publisher to the robot's velocity command topic
    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    size_t count_;
```

```

};

void Stopper::moveStop(){
    auto msg = geometry_msgs::msg::Twist();
    msg.linear.x = FORWARD_SPEED_STOP;
    publisher_ -> publish(msg);
}

void Stopper::moveForward(double forwardSpeed){
    //The default constructor to set all commands to 0
    auto msg=geometry_msgs::msg::Twist();
    //Drive forward at a given speed along the x-axis.
    msg.linear.x = forwardSpeed;
    publisher_ -> publish(msg);
}

void Stopper::moveRight(double turn_right_speed){
    auto msg = geometry_msgs::msg::Twist();
    msg.angular.z = turn_right_speed;
    publisher_ -> publish(msg);
}

void Stopper::startMoving(){
    RCLCPP_INFO(this->get_logger(), "Start moving");
    while (rclcpp::ok()){
        rclcpp::WallRate loop_rate(20); // Create a loop running at 20Hz
        moveForward(FORWARD_SPEED_LOW);
        RCLCPP_INFO(this->get_logger(), "Robot speed: %f", FORWARD_SPEED_LOW);
        auto node = std::make_shared<Stopper>(); // Create a new Stopper node
        rclcpp::spin_some(node); // Spins the node to process incoming ROS messages.
        loop_rate.sleep(); // Sleeps for the remaining time in the 20Hz cycle
    }
}

int main(int argc, char *argv[]){
    rclcpp::init(argc, argv); // Initialize the ROS2 system
    Stopper stopper; // Create an instance of Stopper
    stopper.startMoving(); // Begins the main loop
    return 0; // Indicates successful program termination
}

```

1.4 Building your node.

Before we go to the building step, we should make some necessary changes in [package.xml](#) and [CmakeList.txt](#) files so that system can know how the build the package.

1. Input the following command.

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg
```

2. Using the following command to open the file [package.xml](#).

gedit package.xml

After the line `<buildtool_depend>ament_cmake</buildtool_depend>` add the below lines that required by *buildtool* dependency.

```
<depend>rclcpp</depend>
<depend>std_msgs</depend>
<depend>geometry_msgs</depend>
```

Save the file and exit from it.

- Using the following command to open the file *CMakeLists.txt*.

gedit CMakeLists.txt

Add the following lines below the line *find_package(ament_cmake REQUIRED)*

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
```

Add the following below *add_executable(tutorial_pkg_node src/tutorial_pkg_node.cpp)*.

```
ament_target_dependencies(tutorial_pkg_node rclcpp std_msgs geometry_msgs)
```

Now save your file *CMakeLists.txt*.

- Go to the root of your workspace and build your node.

```
cd ~/M-Drive/ros2_ws
colcon build
```

1.5 Run your node using *ros2 run*

- Now you can use the commands below to run your code.

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
ros2 run tutorial_pkg tutorial_pkg_node
```

You should see the output that prints the robot's velocity in terminal.

```
[INFO] [1639244718.786792287] [Stopper]: Robot speed: 0.100000
[INFO] [1639244718.838366598] [Stopper]: Robot speed: 0.100000
[INFO] [1639244718.890043146] [Stopper]: Robot speed: 0.100000
[INFO] [1639244718.941598425] [Stopper]: Robot speed: 0.100000
[INFO] [1639244718.993213581] [Stopper]: Robot speed: 0.100000
[INFO] [1639244719.044878284] [Stopper]: Robot speed: 0.100000
.....
```

- Use `ctrl^C` to stop the display of the robot's velocity.

1.6 Create a *launch* file to run your code

You have learned how to create a package and how to build and run it via above steps.

Now you can create a launch file to launch multiple nodes, which is a python file.

1. You may use the following commands:

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg
```

2. For the building tool to find the launch file, you need to install it in *CMakeLists.txt* file.

```
gedit CMakeLists.txt
```

You should add the following codes **before** the last line *ament_package()*

```
# Install launch files.
install(DIRECTORY
  launch
  DESTINATION share/${PROJECT_NAME}/
)
```

Save the file and exit.

3. Create a file *tutorial_pkg_node.launch.py* after creating your launch folder.

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg
```

```
mkdir launch
```

```
cd launch
```

```
gedit tutorial_pkg_node.launch.py
```

4. Copy the below code to the new file *tutorial_pkg_node.launch.py* you created.

```
import launch
import launch.actions
import launch.substitutions
import launch_ros.actions

def generate_launch_description():
    return launch.LaunchDescription([
        launch.actions.DeclareLaunchArgument(
            'tutorial_pkg',
            default_value=[launch.substitutions.EnvironmentVariable('USER'), '_'],
            description='tutorial_pkg_node'),
        launch_ros.actions.Node(
            package='tutorial_pkg', executable='tutorial_pkg_node', output='screen',
            name=[launch.substitutions.LaunchConfiguration('tutorial_pkg'),
                'tutorial_pkg_node']),])
```

Save the file and exit.

5. Build your workspace in the root of your workspace.

```
cd ~/M-Drive/ros2_ws
```

```
colcon build
```

6. You can launch your node using the commands below.

```
cd ~/M-Drive/ros2_ws
```

```
source install/setup.sh
```

```
export TURTLEBOT3_MODEL=burger
ros2 launch tutorial_pkg tutorial_node.launch.py
```

You should see the output that print the robot's velocity in the terminal.

```
[tutorial_pkg_node-1] [INFO] [1710518778.425667246] [tutorial_pkg_node]: Start moving
[tutorial_pkg_node-1] [INFO] [1710518778.426135456] [tutorial_pkg_node]: robot speed: 0.10000
[tutorial_pkg_node-1] [INFO] [1710518778.426902351] [tutorial_pkg_node]: robot speed: 0.10000
[tutorial_pkg_node-1] [INFO] [1710518778.447186148] [tutorial_pkg_node]: robot speed: 0.10000
[tutorial_pkg_node-1] [INFO] [1710518778.497365029] [tutorial_pkg_node]: robot speed: 0.10000
.....
```

7. Then you can use `ctrl+C` to stop the display of the robot's velocity.

1.7 How to examine the system.

1. You may open the 1st terminal to launch your node by using either.

```
ros2 run package-name node-name
or
ros2 launch package-name launch-file-name
```

2. Then you can open the second terminal to see the running nodes by using

```
ros2 node list
```

3. In the second terminal, use the following command to see the exact information of a node.

```
ros2 node info node_name
```

4. In the second terminal, use the following command to visualise your system configuration.

```
rqt_graph
```

5. Use the following command to remove build if you want.

```
rm -r build install
```

1.8 Further References if you are interested.

ROS Wiki	https://en.wikipedia.org/wiki/Robot_Operating_System
Installation	https://docs.ros.org/en/humble/Installation.html
Tutorials	https://docs.ros.org/en/humble/Tutorials.html